ГУАП

КАФЕДРА № 43

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

д-р техн. наук, профессор		Ю.А. Скобцов
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №6

Оптимизация путей на графах с помощью муравьиных алгоритмов

по курсу: ЭВОЛЮЦИОННЫЕ МЕТОДЫ ПРОЕКТИРОВАНИЯ ПРОГРАММНО-ИНФОРМАЦИОННЫХ СИСТЕМ

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ ГР. № 4236		А.В. Панченко
	подпись, дата	инициалы, фамилия

Санкт-Петербург 2025

Условие задания с вариантом:

Часть 1

1. Создать программу, использующую МА для решение задачи поиска гамильтонова пути. Индивидуальное заданию выбирается по таблице В.1 в приложении согласно номеру варианта.
2. Представить графически найденное решение. Предусмотреть возможность пошагового просмотра процесса поиска решения.
3. Сравнить найденное решение с представленным в условии задачи оптимальным решением.

Часть 2

Реализовать с использованием муравьиных алгоритмов решение задачи коммивояжера по индивидуальному заданию согласно номеру варианта (см. таблицу 3.1. и приложение Б.).

Представить графически найденное решение

Сравнить найденное решение с представленным в условии задачи оптимальным решением и результатами, полученными в лабораторной работе №3..

Проанализировать время выполнения и точность нахождения результата в зависимости от вероятности различных видов кроссовера, мутации.

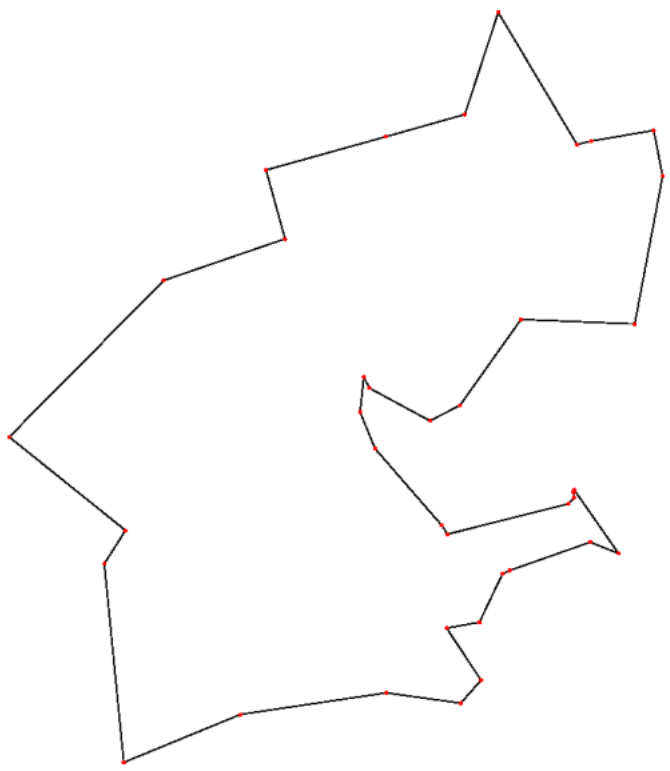
dj89: 89 городов в Джибути

[Ошибка! Источник ссылки не найден.]

Тип данных: эвклидовы координаты городов

```
1 11511.3889 42106.3889
2 11503.0556 42855.2778
3 11438.3333 42057.2222
4 11438.3333 42057.2222
5 11438.3333 42057.2222
6 11785.2778 42884.4444
7 11785.2778 42884.4444
8 11785.2778 42884.4444
9 11785.2778 42884.4444
10 12363.3333 43189.1667
11 11846.9444 42660.5556
12 11503.0556 42855.2778
13 11963.0556 43290.5556
14 11963.0556 43290.5556
15 12300.0000 42433.3333
16 11973.0556 43026.1111
17 11973.0556 43026.1111
18 11461.1111 43252.7778
19 11461.1111 43252.7778
20 11461.1111 43252.7778
21 11461.1111 43252.7778
22 11600.0000 43150.0000
23 12386.6667 43334.7222
24 12386.6667 43334.7222
25 11595.0000 43148.0556
26 11595.0000 43148.0556
27 11569.4444 43136.6667
28 11310.2778 42929.4444
29 11310.2778 42929.4444
30 11310.2778 42929.4444
31 11963.0556 43290.5556
32 11416.6667 42983.3333
33 11416.6667 42983.3333
34 11595.0000 43148.0556
35 12149.4444 42477.5000
36 11595.0000 43148.0556
37 11595.0000 43148.0556
38 11108.6111 42373.8889
39 11108.6111 42373.8889
40 11108.6111 42373.8889
41 11108.6111 42373.8889
42 11183.3333 42933.3333
43 12372.7778 42711.3889
44 11583.3333 43150.0000
45 11583.3333 43150.0000
46 11583.3333 43150.0000
47 11583.3333 43150.0000
48 11583.3333 43150.0000
49 11822.7778 42673.6111
50 11822.7778 42673.6111
51 12058.3333 42195.5556
52 11003.6111 42102.5000
53 11003.6111 42102.5000
54 11003.6111 42102.5000
55 11522.2222 42841.9444
56 12386.6667 43334.7222
57 12386.6667 43334.7222
58 12386.6667 43334.7222
59 11569.4444 43136.6667
60 11569.4444 43136.6667
61 11569.4444 43136.6667
62 11155.8333 42712.5000
63 11155.8333 42712.5000
64 11155.8333 42712.5000
65 11155.8333 42712.5000
66 11133.3333 42885.8333
67 11133.3333 42885.8333
68 11133.3333 42885.8333
69 11133.3333 42885.8333
70 11133.3333 42885.8333
71 11003.6111 42102.5000
72 11770.2778 42651.9444
73 11133.3333 42885.8333
74 11690.5556 42686.6667
75 11690.5556 42686.6667
76 11751.1111 42814.4444
77 12645.0000 42973.3333
78 12421.6667 42895.5556
79 12421.6667 42895.5556
80 11485.5556 43187.2222
81 11423.8889 43000.2778
82 11423.8889 43000.2778
83 11715.8333 41836.1111
84 11297.5000 42853.3333
85 11297.5000 42853.3333
86 11583.3333 43150.0000
87 11569.4444 43136.6667
88 12286.9444 43355.5556
89 12355.8333 43156.3889
EOF
```

Оптимальный тур dj89:



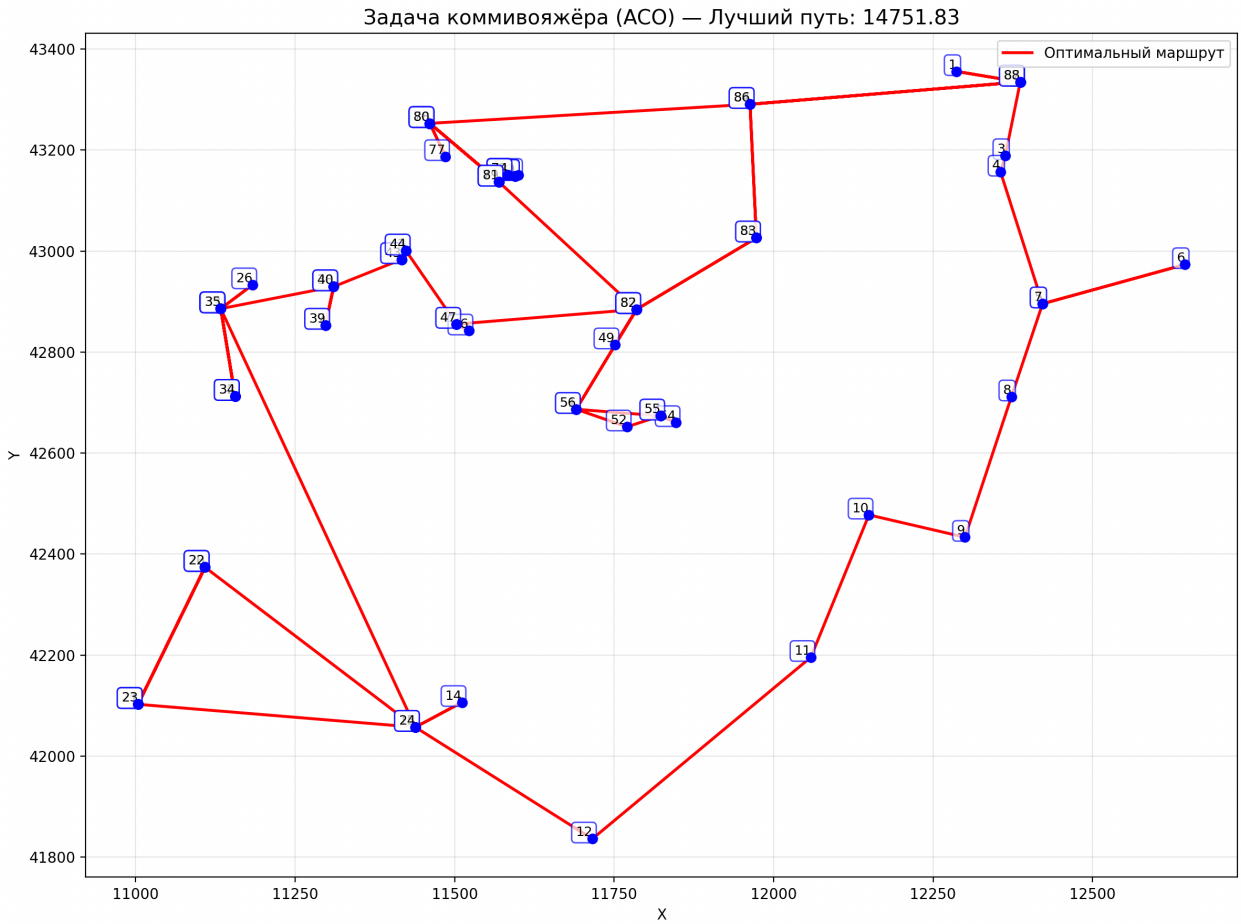
Распечатанный листинг программы:

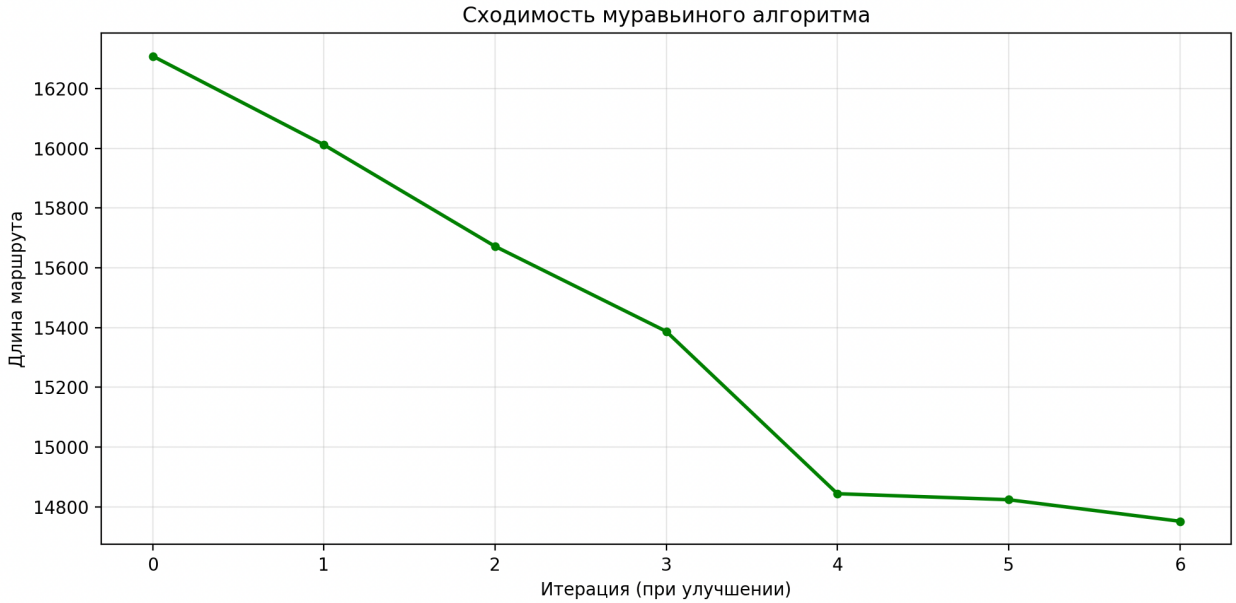
```
import numpy as np
import random
import matplotlib.pyplot as plt

# Координаты городов (89 городов из dj89.tsp)
coordinates = [
    (11511.3889, 42106.3889), (11503.0556, 42855.2778), (11438.3333, 42057.2222),
    (11438.3333, 42057.2222), (11438.3333, 42057.2222), (11785.2778, 42884.4444),
    (11785.2778, 42884.4444), (11785.2778, 42884.4444), (11785.2778, 42884.4444),
    (12363.3333, 43189.1667), (11846.9444, 42660.5556), (11503.0556, 42855.2778),
    (11963.0556, 43290.5556), (11963.0556, 43290.5556), (12300.0000, 42433.3333),
    (11973.0556, 43026.1111), (11973.0556, 43026.1111), (11461.1111, 43252.7778),
    (11461.1111, 43252.7778), (11461.1111, 43252.7778), (11461.1111, 43252.7778),
    (11600.0000, 43150.0000), (12386.6667, 43334.7222), (12386.6667, 43334.7222),
    (11595.0000, 43148.0556), (11595.0000, 43148.0556), (11569.4444, 43136.6667),
    (11310.2778, 42929.4444), (11310.2778, 42929.4444), (11310.2778, 42929.4444),
    (11963.0556, 43290.5556), (11416.6667, 42983.3333), (11416.6667, 42983.3333),
    (11595.0000, 43148.0556), (12149.4444, 42477.5000), (11595.0000, 43148.0556),
    (11595.0000, 43148.0556), (11108.6111, 42373.8889), (11108.6111, 42373.8889),
    (11108.6111, 42373.8889), (11108.6111, 42373.8889), (11183.3333, 42933.3333),
    (12372.7778, 42711.3889), (11583.3333, 43150.0000), (11583.3333, 43150.0000),
    (11583.3333, 43150.0000), (11583.3333, 43150.0000), (11583.3333, 43150.0000),
    (11822.7778, 42673.6111), (11822.7778, 42673.6111), (12058.3333, 42195.5556),
    (11003.6111, 42102.5000), (11003.6111, 42102.5000), (11003.6111, 42102.5000),
    (11522.2222, 42841.9444), (12386.6667, 43334.7222), (12386.6667, 43334.7222),
    (12386.6667, 43334.7222), (11569.4444, 43136.6667), (11569.4444, 43136.6667),
    (11569.4444, 43136.6667), (11155.8333, 42712.5000), (11155.8333, 42712.5000),
    (11155.8333, 42712.5000), (11155.8333, 42712.5000), (11133.3333, 42885.8333),
    (11133.3333, 42885.8333), (11133.3333, 42885.8333), (11133.3333, 42885.8333),
    (11133.3333, 42885.8333), (11003.6111, 42102.5000), (11770.2778, 42651.9444),
    (11133.3333, 42885.8333), (11690.5556, 42686.6667), (11690.5556, 42686.6667),
    (11751.1111, 42814.4444), (12645.0000, 42973.3333), (12421.6667, 42895.5556),
    (12421.6667, 42895.5556), (11485.5556, 43187.2222), (11423.8889, 43000.2778),
    (11423.8889, 43000.2778), (11715.8333, 41836.1111), (11297.5000, 42853.3333),
    (11297.5000, 42853.3333), (11583.3333, 43150.0000), (11569.4444, 43136.6667),
    (12286.9444, 43355.5556), (12355.8333, 43156.3889)
]

n = len(coordinates)  # Количество городов

# Вычисление матрицы расстояний
dist = np.zeros((n, n))
for i in range(n):
    for j in range(n):
        if i != j:
            dx = coordinates[i][0] - coordinates[j][0]
            dy = coordinates[i][1] - coordinates[j][1]
            dist[i][j] = np.sqrt(dx*dx + dy*dy)

# Параметры муравьиного алгоритма (SACO)
alpha = 1.0      # Влияние феромона
```

```python
beta = 5.0       # Влияние эвристики (1/расстояние)
rho = 0.5        # Коэффициент испарения феромона
Q = 100          # Константа откладывания феромона
num_ants = 50    # Количество муравьёв
max_iter = 200   # Максимальное число итераций
tau0 = 1e-6      # Начальная концентрация феромона

# Инициализация феромона
tau = np.full((n, n), tau0)

# Список для графика сходимости
history = []

# Функция выбора следующего города (формула 6.1)
def choose_next_city(current, visited, tau, dist):
    candidates = [city for city in range(n) if city not in visited]
    if not candidates:
        return None

    numerators = []
    for next_city in candidates:
        pheromone = tau[current][next_city]
        distance = dist[current][next_city]
        heuristic = 0.0 if distance == 0 else 1.0 / distance
        numerators.append(pheromone ** alpha * heuristic ** beta)

    total = sum(numerators)
    if total == 0:
        return random.choice(candidates)

    probabilities = [num / total for num in numerators]
    return np.random.choice(candidates, p=probabilities)

# Основной цикл муравьиного алгоритма
best_tour = None
best_length = float('inf')

print("Запуск муравьиного алгоритма...")
for iteration in range(max_iter):
    ant_tours = []
    ant_lengths = []

    for ant in range(num_ants):
        start = random.randint(0, n-1)
        tour = [start]
        visited = set([start])
        current = start

        while len(tour) < n:
            next_city = choose_next_city(current, visited, tau, dist)
            if next_city is None:
                break
            tour.append(next_city)
```

```
            visited.add(next_city)
            current = next_city

        if len(tour) == n:
            tour.append(start)
            length = sum(dist[tour[i]][tour[i+1]] for i in range(n))
            ant_tours.append(tour)
            ant_lengths.append(length)

    # Обновление лучшего решения
    if ant_lengths:
        min_idx = np.argmin(ant_lengths)
        current_best = ant_lengths[min_idx]
        if current_best < best_length:
            best_length = current_best
            best_tour = ant_tours[min_idx][:]
            history.append(best_length)  # Запоминаем улучшение

    # Испарение феромона
    tau *= (1 - rho)

    # Откладывание феромона
    for k in range(len(ant_tours)):
        tour = ant_tours[k]
        length = ant_lengths[k]
        delta_tau = Q / length
        for i in range(n):
            u = tour[i]
            v = tour[i+1]
            tau[u][v] += delta_tau
            tau[v][u] += delta_tau

    # Вывод прогресса
    if iteration % 20 == 0 or iteration == max_iter - 1:
        print(f"Итерация {iteration:3d} | Лучшая длина: {best_length:8.2f}")

# Результаты
print("\n" + "="*60)
print(f"АЛГОРИТМ ЗАВЕРШЁН")
print(f"Лучшая длина маршрута: {best_length:.2f}")
print(f"Маршрут: {[int(x) for x in best_tour]}")
print("="*60)

# График маршрута
plt.figure(figsize=(12, 9))
x = [coordinates[i][0] for i in best_tour]
y = [coordinates[i][1] for i in best_tour]
plt.plot(x, y, 'r-', linewidth=2, label='Оптимальный маршрут')
plt.scatter(x[:-1], y[:-1], c='blue', s=40, zorder=5)
for i, (xi, yi) in enumerate(zip(x[:-1], y[:-1])):
    plt.text(xi, yi, str(i), fontsize=9, ha='right', va='bottom',
bbox=dict(boxstyle="round", fc="white", ec="blue", alpha=0.7))
plt.title(f"Задача коммивояжёра (ACO) — Лучший путь: {best_length:.2f}", fontsize=14)
```

```python
plt.xlabel("X")
plt.ylabel("Y")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig("route.png", dpi=300, bbox_inches='tight')
plt.show()

# График сходимости
if len(history) > 1:
    plt.figure(figsize=(10, 5))
    plt.plot(history, 'g-', linewidth=2, marker='o', markersize=4)
    plt.title("Сходимость муравьиного алгоритма")
    plt.xlabel("Итерация (при улучшении)")
    plt.ylabel("Длина маршрута")
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.savefig("convergence.png", dpi=300, bbox_inches='tight')
    plt.show()
else:
    print("Сходимость не улучшалась — график не построен.")
```





Запуск муравьиного алгоритма...

Итерация 0 | Лучшая длина: 16308.46

Итерация 20 | Лучшая длина: 14843.94

Итерация 40 | Лучшая длина: 14823.86

Итерация 60 | Лучшая длина: 14823.86

Итерация 80 | Лучшая длина: 14823.86

Итерация 100 | Лучшая длина: 14823.86

Итерация 120 | Лучшая длина: 14823.86

Итерация 140 | Лучшая длина: 14823.86

Итерация 160 | Лучшая длина: 14751.83

Итерация 180 | Лучшая длина: 14751.83

Итерация 199 | Лучшая длина: 14751.83

==========

АЛГОРИТМ ЗАВЕРШЁН

Лучшая длина маршрута: 14751.83

Маршрут: [57, 87, 55, 9, 88, 77, 76, 78, 42, 14, 34, 50, 82, 3, 0, 2, 39, 70, 40, 51, 37, 53, 38, 52, 4, 72, 41, 69, 62, 66, 61, 67, 64, 68, 63, 65, 29, 83, 28, 84, 27, 32, 81, 31, 80, 1, 54, 11, 6, 75, 8, 73, 71, 48, 10, 49, 74, 5, 15, 30, 18, 60, 44, 35, 21, 24, 46, 25, 47, 33, 43, 36, 85, 26, 45, 86, 19, 79, 20, 58, 17, 59, 7, 16, 13, 23, 12, 56, 22, 57]

==========

Вывод:

В ходе выполнения лабораторной работы был реализован простой муравьиный алгоритм для решения задачи коммивояжёра, основанный на принципах кооперации искусственных муравьёв, откладывающих феромон на рёбрах графа. Алгоритм успешно моделирует процесс поиска кратчайшего замкнутого маршрута, учитывая как накопленный феромон, так и эвристическую информацию о расстояниях между городами. Реализованы этапы инициализации феромона, построения решений каждым муравьём, испарения феромона и его обновления в зависимости от качества найденных путей. Полученное решение визуализировано графически с отображением оптимального маршрута и динамики сходимости, что позволило наглядно продемонстрировать эффективность метода в задачах комбинаторной оптимизации.